



PDI: ATIVIDADE 03 · REGISTRO DE ENTREGA

Nós Customizados e Expressões Avançadas n8n

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

n8n Enterprise / code nodes JS·Python / expressões avançadas / payload pesado

1. Contexto

Os pipelines n8n da V4 Company (SDR IA e MarTech) processam payloads cada vez maiores: listas de leads, sincronizações de CRM, importações em massa. Os nós `Code` que fazem esse trabalho foram escritos como "um nó que faz tudo": carregam o payload inteiro em memória, re-parseiam JSON a cada etapa, percorrem listas com complexidade $O(n^2)$ e repetem a mesma lógica entre dezenas de workflows. Três problemas estruturais motivam este PDI:

- **Payloads pesados estouram a instância:** 100k+ itens com `map().filter()` encadeado e `{...item}` por iteração geram OOM e event loop bloqueado.
- **Parse e complexidade desperdiçados:** `JSON.parse` dentro de loops e buscas por item (`.find, indexOf`) viram $O(n^2)$.
- **Lógica duplicada e sem padrão:** o mesmo transform copiado entre workflows, expressões inline difíceis de manter e nenhuma biblioteca compartilhada.

2. Diagnóstico

2.1 Causa raiz

Não existe uma **camada padrão de transformação** no n8n da V4. Cada nó Code reinventa a roda: e a forma "mais fácil de escrever" é a mais cara de rodar. O resultado é o sintoma já visto na atividade 1 (ADPLAN: JS timeout de 25 min) e na atividade 2 (payload pesado sem processamento incremental).

3. Solução em 3 frentes

FRENTE	SOLUÇÃO	WORKFLOW
1. Nós Code JS avançados	Streaming com chunking, dedupe O(n) por chave primitiva, cópia mínima de objetos e parse 1x na entrada.	[CC] NOS - JS Payload Normalizer
2. Nós Code Python no n8n	Enriquecimento e agregação pesados com a biblioteca padrão (collections, itertools): sem dependências externas.	[CC] NOS - Python Payload Enricher
3. Expressões avançadas + biblioteca	Referências entre nós, JSONata, condicionais, memoização e uma biblioteca única 3-lib/ como fonte da verdade.	[CC] NOS - Expressions & Memo Playground

```
// O coração da mudança: de O(n²) para O(n)
// ANTES: busca por item dentro do loop
items.forEach(item => { const x = items.find(o => o.id === item.ref); })

// DEPOIS: índice Map + Set de dedupe: UMA passada
const seen = new Set();
for (const item of items) {
  if (seen.has(item.id)) continue;
  seen.add(item.id);
  out.push({ json: normalize(item) });
}
```

4. Pipeline de processamento

ETAPA	RESPONSÁVEL	TÉCNICA
-------	-------------	---------

1. Entrada	<code>Webhook /nos/js-normalizer</code>	Payload bruto (string ou array)
2. Parse + Chunk	<code>Parse and Chunk</code> (JS)	Parse 1x + chunking streaming (default 1000)
3. Normalizar	<code>Normalize in One Pass</code> (JS)	Filtra + dedupe + transforma em O(n)
4. Enriquecer	<code>Enrich Python</code> (Python)	Agregação stdlib (Counter/defaultdict)
5. Métricas	<code>Return Metrics</code> (JS)	durationMs + items/s + dedupe

Como a engenharia funciona: **payload pesado de ponta a ponta**

● FLUXO EM EXECUÇÃO

1

Payload pesado (10k–100k itens)

Chega via webhook. Antes, `map().filter()` encadeado e cópias por item geravam OOM e event loop bloqueado.

10k+ itens

OOM no padrão antigo

2

Parse + Chunk (JS)

Decodifica o JSON 1x na entrada e quebra em chunks de 1000. Nunca re-parseia dentro do loop.

parse 1x

chunk 1000

3

Normalize in One Pass (JS)

Filtra, dedupe (Set por chave primitiva) e transforma no **mesmo** loop: complexidade O(n), cópia mínima de campos.

O(n)

dedupe Set

cópia mínima

4

Enrich Python (Python)

Enriquecimento e agregação com a biblioteca padrão (`collections`, `itertools`): sem pandas, sem pip.

stdlib only

Counter/defaultdict

5

Return Metrics (JS)

Entrega `processedItems`, `deduped`, `durationMs` e `itemsPerSecond`: a base das queries de performance do `5-monitoring/`.

items/s

dedupe ratio

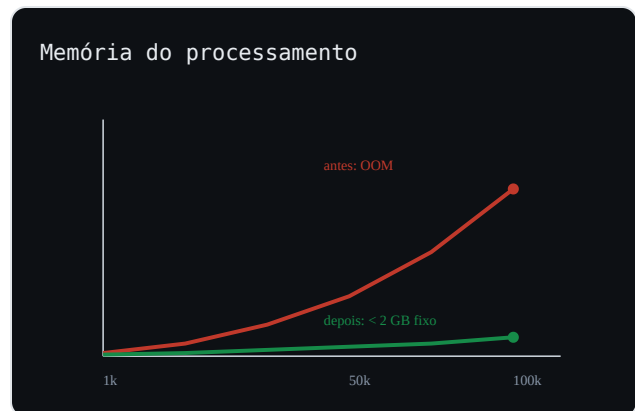
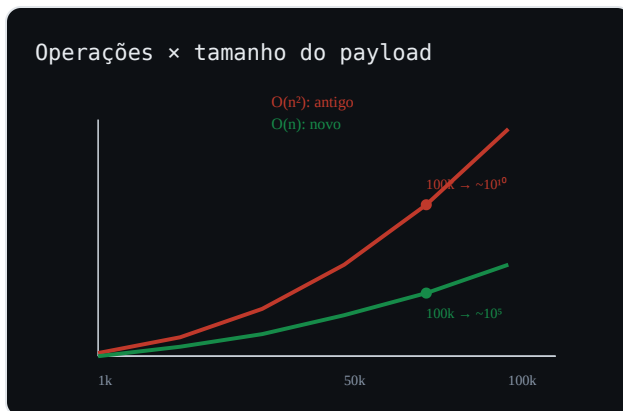
Cada item atravessa uma única vez: a engrenagem processa em stream, não em lotes de memória.

● parse 1x ● stream ● $O(n)$ ● metrics

Escala: o mesmo fluxo que processa 10 itens processa 100k: o chunking streaming e o dedupe $O(n)$ garantem memória controlada e event loop livre.

5. Por que $O(n)$ muda tudo

O mesmo payload de 100 mil itens custa bilhões de operações no padrão antigo ($O(n^2)$) e poucas centenas de milhares no novo ($O(n)$): e a memória acompanha:



Chunking streaming mantém o pico de memória **constante**: o payload inteiro nunca fica residente. É por isso que o mesmo nó processa 10 ou 100 mil itens sem trocar de máquina.

EXPLICAÇÃO DIDÁTICA

Imagine uma esteira com 100 mil caixas. No jeito antigo, para cada caixa a pessoa voltava e conferia todas as outras: o processo travava. No novo, cada caixa passa uma única vez e você anota numa lista quem já viu: a esteira nunca para, mesmo com 100 mil caixas.

6. Memoização entre execuções

Valores estáveis (limiar, taxa, config) são cacheados com `$getWorkflowStaticData('global')`: calculados 1x e reutilizados nas execuções seguintes, sem repetir chamadas caras.

```
// Memoize Reference: cache entre execuções
const staticData = $getWorkflowStaticData('global');
```

```
if (staticData.scoreThreshold === undefined) {
  staticData.scoreThreshold = 70; // busca cara feita 1x
}

// Segunda execução: cachedAt NÃO muda
```

7. Biblioteca reutilizável 3-lib

ARQUIVO	FUNÇÕES
payload-lib.js	chunk · normalizeStream · dedupe · aggregate · memoizeGlobal · toOutput
payload-lib.py	chunk · dedupe · aggregate · parse_payload · to_output

Fonte da verdade: nunca editar um nó Code sem atualizar a lib. Os workflows embutem cópias das funções usadas; a lib é o padrão único.

8. Workflows entregues

WORKFLOW	TIPO	FUNÇÃO
[CC] NOS - JS Payload Normalizer	Webhook /nos/js-normalizer	Parse 1x, chunking streaming, dedupe e normalização O(n)
[CC] NOS - Python Payload Enricher	Webhook /nos/python-enricher	Enriquecimento/agregação com stdlib (set + Counter)
[CC] NOS - Expressions & Memo Playground	Manual	Expressões avançadas + memoização (\$getWorkflowStaticData)

9. Métricas de sucesso

MÉTRICA	ANTES	META
Payload 100k itens	OOM / timeout	Streaming, < 2 GB pico

Complexidade de normalização	$O(n^2)$	$O(n)$ padrão
Re-parse de JSON por pipeline	Múltiplos por etapa	1x na entrada
Código duplicado entre workflows	Alto (copiar/colar)	Biblioteca única 3-lib
Expressões de manutenção difícil	Inline, não testáveis	Padronizadas + JSONata

10. Entregas

PASTA	CONTEÚDO	FORMATO
1-standards/	3 padrões: nós Code JS, Python no n8n, expressões avançadas	Markdown
2-workflows/	3 workflows validados com n8nac	.workflow.ts
3-lib/	Biblioteca reutilizável payload-lib (JS + Python)	JS + Python
4-retrofit/	Como adaptar nós Code existentes (ADPLAN, PRO ANÁLISES, CC)	Markdown
5-monitoring/	Queries de performance + alertas de duração/dedupe	Markdown
6-automation/	Deploy com gate de aprovação (dry-run)	Shell
7-apresentacao/	Deck HTML + demo + este relatório	HTML + Markdown
MANUAL - IMPLEMENTACAO.md	Manual de implementação completo (6 fases + rollback + checklist de homologação)	Markdown

11. Decisões e tradeoffs

Escolhas derivadas do conteúdo real da atividade, com custo e benefício explícitos.

- **Parse 1x na entrada com chunk default de 1000:** elimina re-parse dentro de loop. Exige padronizar a entrada como string ou array no Webhook /nos/js-normalizer.
- **Normalização $O(n)$ com Set por chave primitiva:** payload de 100k itens sai de bilhões de operações para centenas de milhares. Exige cópia mínima de campos em vez de spread completo.

- **Python só com stdlib (collections, itertools, Counter e defaultdict):** roda no Code node sem dependência externa. Menos sintaxe pronta em troca de deploy simples.
- **Biblioteca única 3-lib como fonte da verdade:** funções chunk, dedupe, aggregate, normalizeStream, memoizeGlobal e toOutput num lugar só. Workflows embutem cópias e exigem sincronizar a cada mudança.
- **Memoização com \$getWorkflowStaticData global:** limiar e config calculados 1x e reutilizados. Dado cacheado pode defasar e exige invalidação consciente.

12. Impacto no negócio

O sintoma já visto na atividade 1 com JS timeout de 25 min no ADPLAN e na atividade 2 com payload sem processamento incremental travava importações e sincronizações ao escalar de 10k para 100k itens. Com streaming, dedupe $O(n)$ e pico abaixo de 2 GB, o mesmo fluxo processa 10 ou 100k itens sem trocar de máquina, com parse único e código único na 3-lib em vez de cópia entre dezenas de workflows. Isso corta tempo de execução, elimina OOM e event loop bloqueado e reduz custo de manutenção de expressões inline não testáveis para padrão com JSONata.

13. Referências de estudo

Material para aprofundar os temas usados nesta atividade.

- **Curso:** JavaScript Performance, estruturas de dados e complexidade, na Alura.
- **Vídeo:** JavaScript Event Loop and Memory Explained, no YouTube, canal Fireship.
- **Doc:** n8n Docs, Code node JavaScript and Python, na plataforma n8n Docs.
- **Doc:** MDN Docs, Map, Set and Array iteration, na plataforma MDN Web Docs.

Status: desenvolvido · workflows validados e prontos para produção · aguardando homologação.
Nenhum workflow foi publicado no n8n nesta etapa. Implementar seguindo o MANUAL - IMPLEMENTACAO.md na raiz da atividade.